

Audio Tokens for Multimodal LLMs

Mirco Ravanelli



Montreal AI Ecosystem

Academia



Public Agencies



Companies



Montreal AI Ecosystem



- **A young university (founded just 50 years ago)**
- **Canada's fastest-rising research universities.**
- **Major strategic investments in AI, cybersecurity, and robotics.**
- **Deeply integrated into Montréal's AI.**



- **Founded by Yoshua Bengio, Turing Award laureate**
- **One of the largest deep learning research centers in the world**
- **1,000+ researchers (students, faculty, postdocs, collaborators)**
- **World-leading expertise in deep learning.**
- **Strong industry partnerships (Meta, Google, Microsoft, Samsung).**

Outline

- Text-only LLMs
- MultiModal LLMs
- Acoustic Tokens
- Semantic Tokens
- Hybrid Tokens
- Benchmarks
- Open Challenges and Future Directions

Text-Only LLMs

Text-Only LLMs

- Language Models have a **long history**.



Predict the **next word** from the **previous ones**.

I like ragù alla *bolognese* (of course)



N-grams



MLPs



RNNs



Transformers

1980~2010

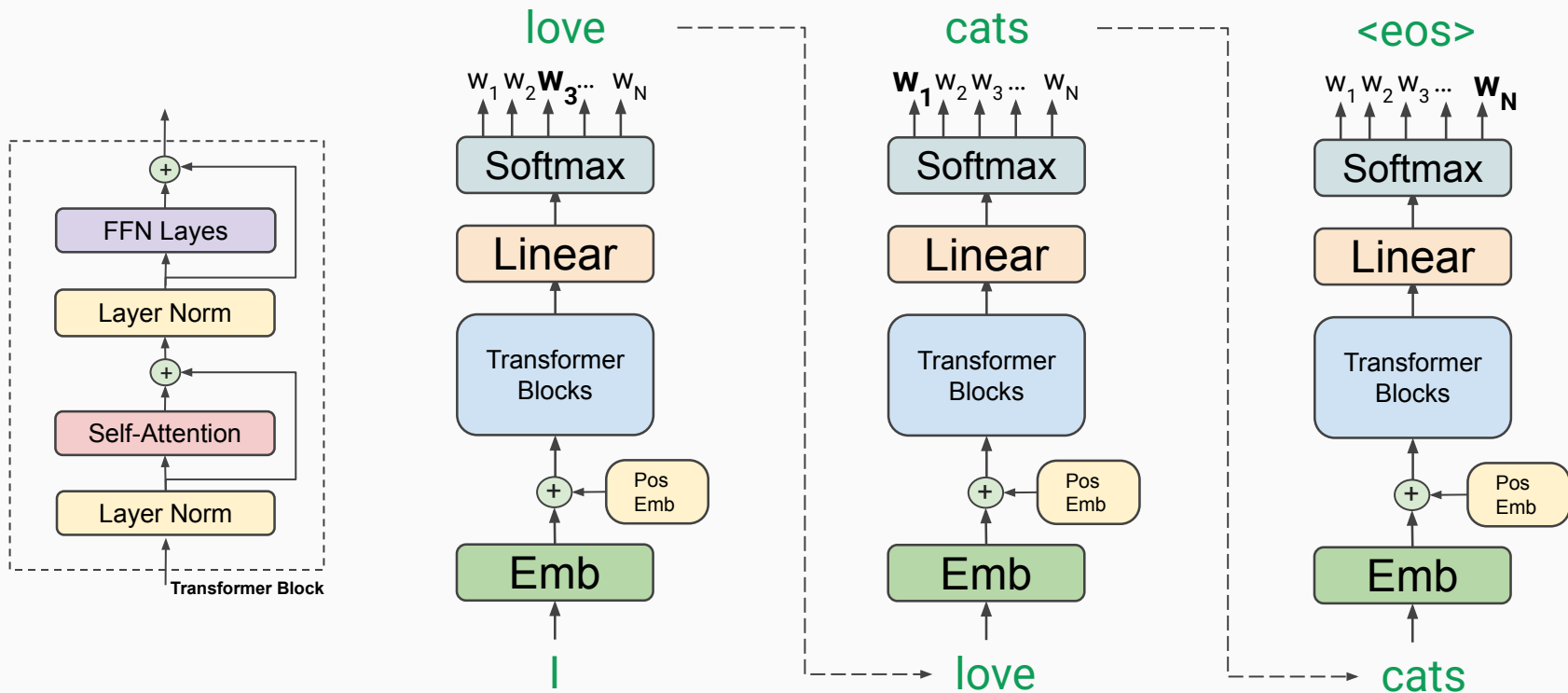
2002~2010

2010~2018

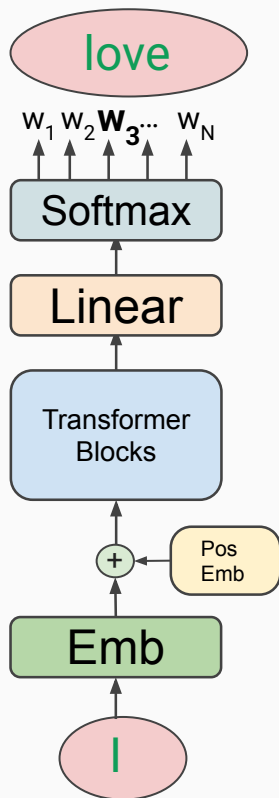
2018~Now

Text-Only LLMs

- **Dominant Approach:** Autoregressive Transformers



Text Tokens



- We don't actually use **words** as the basic units!

WHY?



The system cannot generalize to new words.

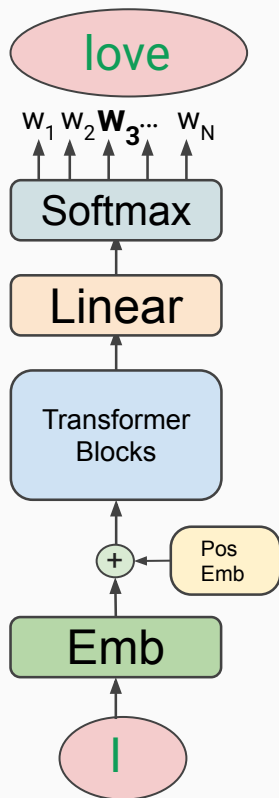


Rare words might have few training examples.



Large output space.

Text Tokens



- What if we use **characters** instead?



Small output space (we might have 20-30 chars).



The system can generalize to new words.



Enough training material for each character.



Long Sequences

- More memory (Quadratic bottleneck).
- More computation.
- Harder training.

Text Tokens



What if we use something between words and characters?

- **Core concept:** Create tokens for **common character sequences**.
- For instance:

```
"THE CITY OF MONTREAL" => ['THE', '_CITY', '_OF', '_MO', 'NT', 'RE', 'AL']
```



Sequences stay reasonably short.



Can generalize to new words.



Each token has enough training examples.

LLM Training

- **Train LLMs:** predict the next token across massive text.



Scale Matters!

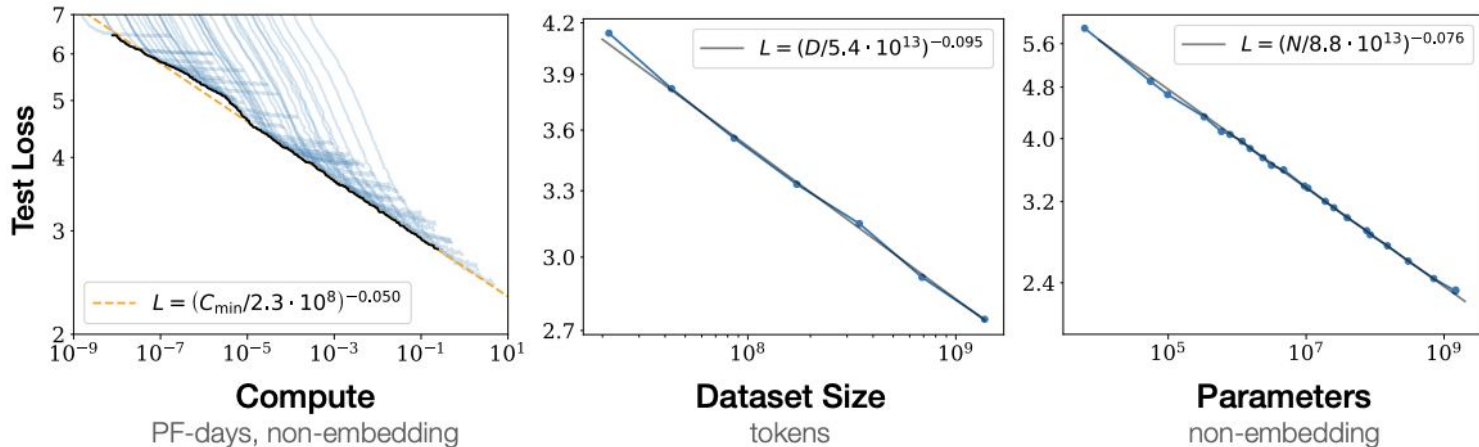


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Beyond Text-Only LLMs

Limitations of Text-Only LLMs

- **Superficial Understanding of the World**

Weak Generalization Higher Risk of Hallucinations



- **Rigid Human–Machine Interaction**

Humans communicate with speech, gestures, images, and sounds.



Multimodal LLMs!

Multimodal LLMs

- A **multimodal LLM** is a model that **understands** and **generates** information across multiple modalities such as **text**, **audio**, and **images**.



Deeper Understanding



Fewer Hallucinations



Better Generalization



Flexible Human-Machine Interaction

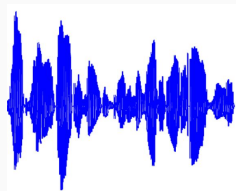
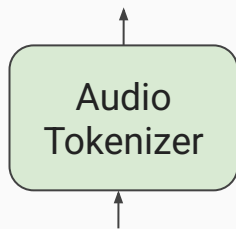
Multimodal LLMs



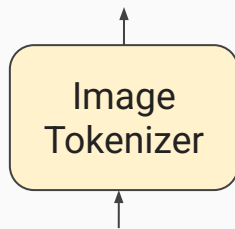
How Do We Make an LLM Multimodal?

Dominant Approach: Convert All Modalities (*Text, Images, Audio*) into **Tokens**.

[14, 64, 76, 67, 67, 89]

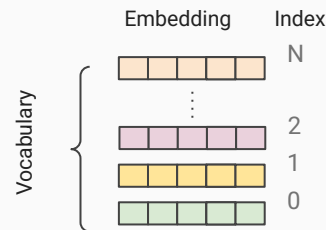


[14, 64, 76, 67, 67, 89]



We convert continuous signals (e.g, audio and images) into a finite **sequence** of **discrete indexes**.

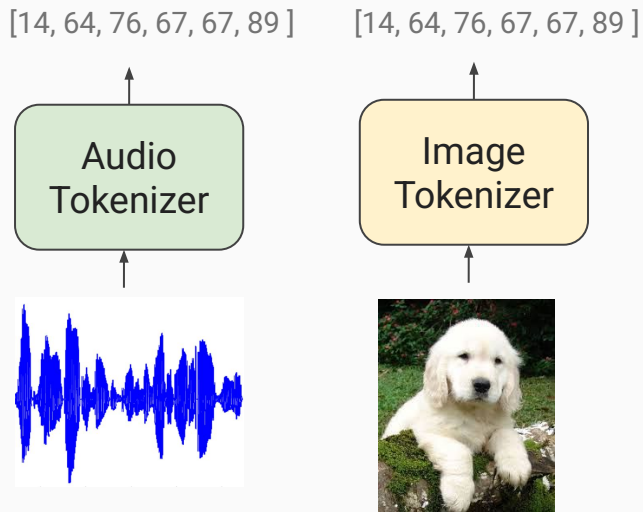
Each index corresponds to a learned **continuous embedding vector**.



Each token aims to capture a **part of the information** contained in the original signal.

Multimodal LLMs

- Learning good tokens is **challenging** for audio and images!



WHY?

Audio and images are **rich continuous signals!**

Loss of Information

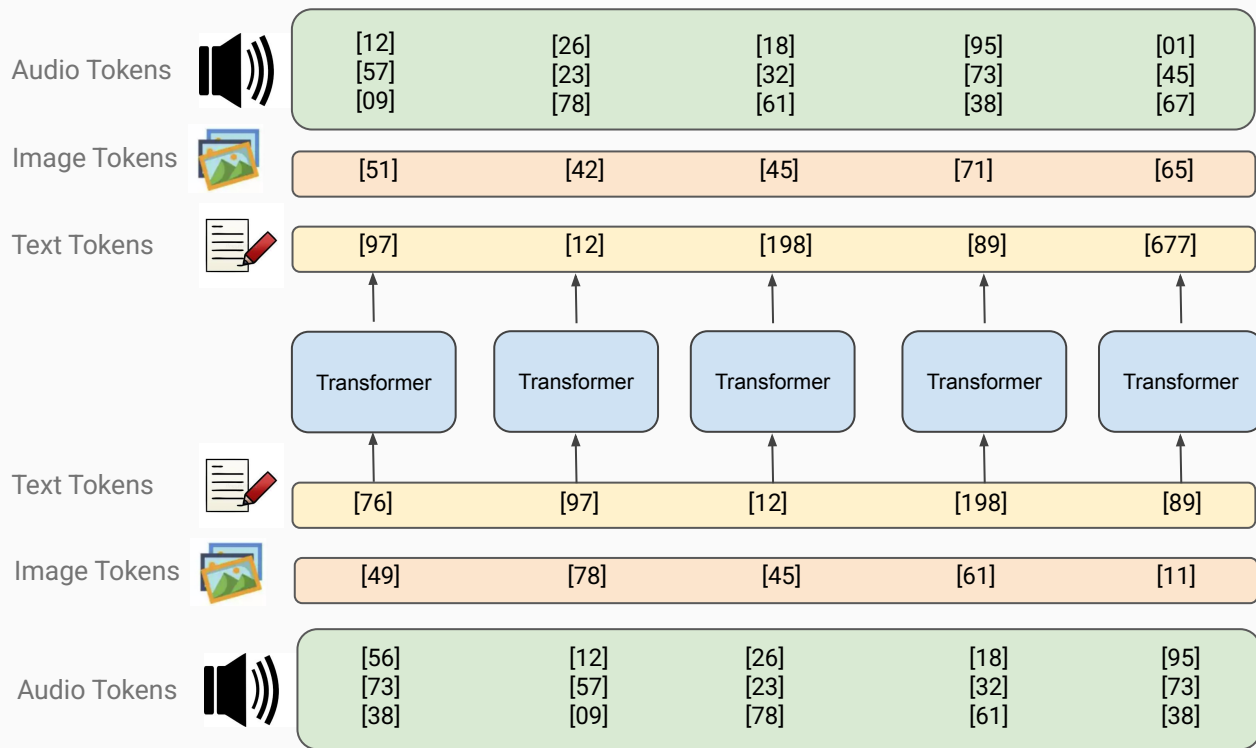
Capturing all relevant details with compact tokens is challenging.

Long Sequences

Images and audio produce far more tokens than text, increasing model size and compute.

Multimodal LLMs

- We can jointly process different tokens into a single autoregressive **Multistream Transformer**.



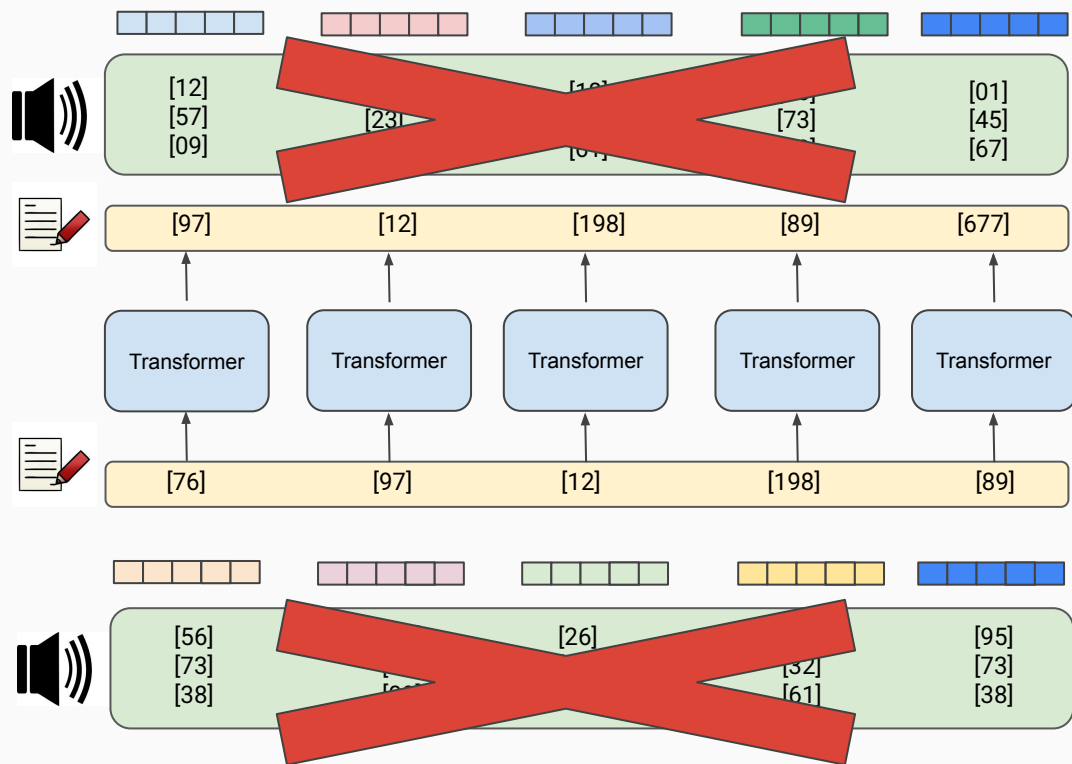
- **Simple extension** of standard text-only LLMs.
- **Multiple streams** handle different modalities.
- For audio, multiple token streams are often needed due to complexity.
- Learned **decoders** enable the generation of audio and images from audio tokens.

Multimodal LLMs



Multimodal LLMs

- Wait... but *why bother with tokens?* Why not just use **continuous representations**?



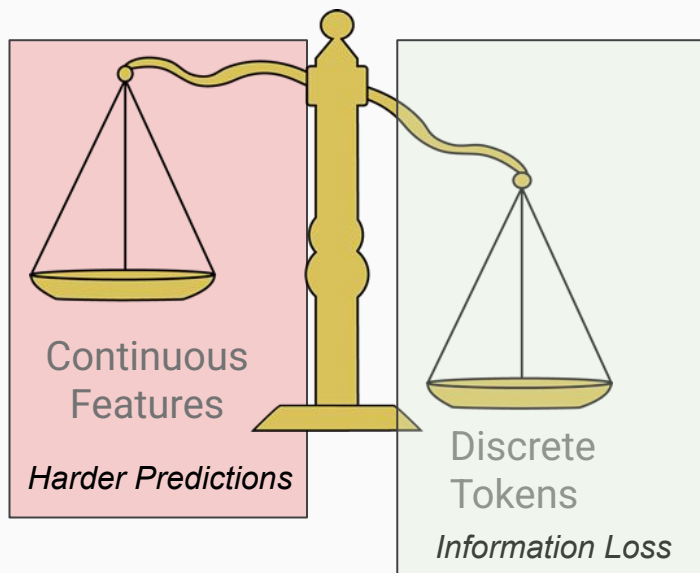
We already have excellent continuous speech features:

- FilterBanks**
- Features from self-supervised models such as **wav2vec**, **WavLM**, **HuBERT**, etc.
- Self-supervised continuous features achieve **state-of-the-art performance** in many speech tasks.

Discretization can lead to **information loss**.

Multimodal LLMs

- To understand why we prefer discrete tokens over continuous features, we need to consider the following **trade-off**:



Continuous Features:

- Predicting continuous outputs is **harder** than predicting discrete tokens.
- It requires **Mean Squared Error (MSE)** instead of **Negative Log-Likelihood (NLL)**
- MSE is notoriously **difficult** to **optimize**.

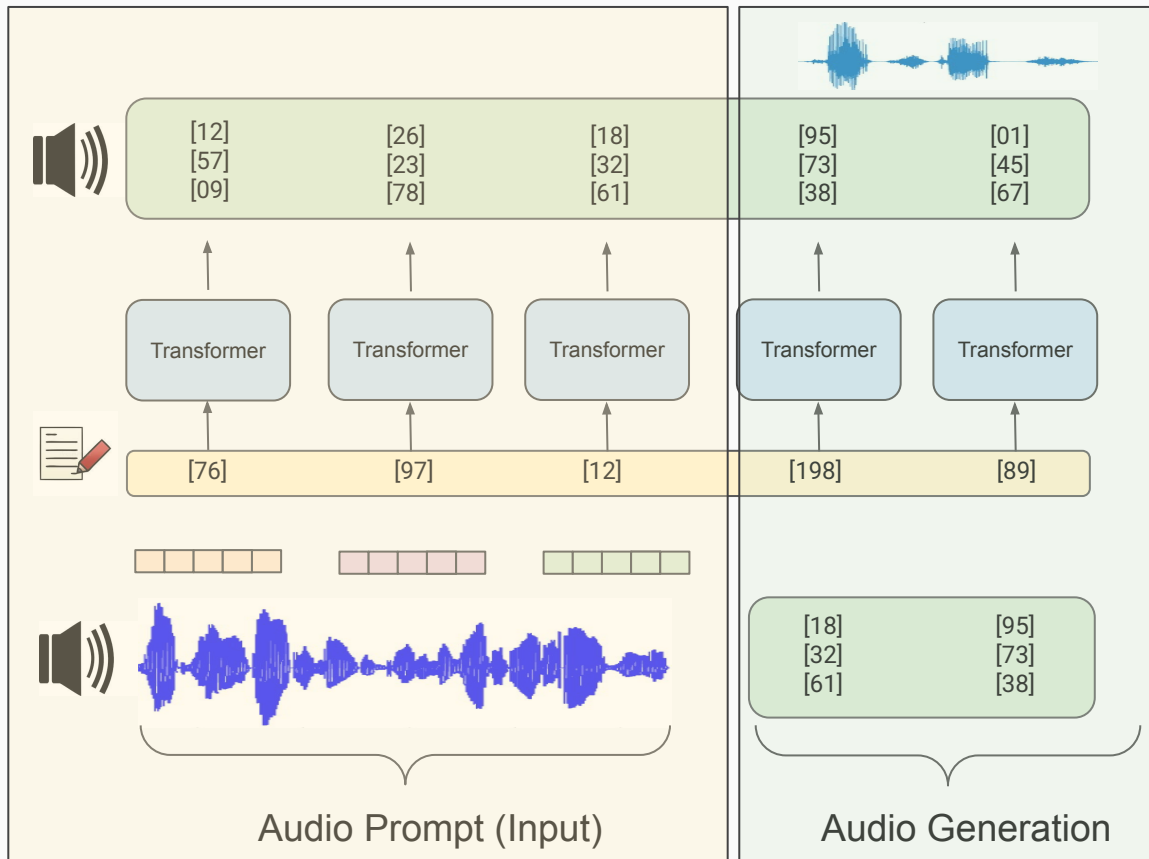
Discrete Tokens:

- Discrete tokens **lose some information**.
- Improved **learning dynamics** and **training stability**.



Currently, the learning difficulties caused by predicting continuous outputs **outweigh** the loss from discretization.

Multimodal LLMs



- Continuous representations are often used for the **input audio/image prompts** only in modern multimodal LLMs.
- All the generation part, instead, is based on **discrete tokens**.

Audio Tokens

Audio Tokens

- We have seen that tokenizing continuous signals is **difficult** and introduces **information loss**.
- For **speech**, this is even **more challenging**:

WHY?

Speech carries multiple layers of information at once!



Words

Phonemes

Accent

*Speaker
Identity*

Emotions

....



Audio Tokens

Ideal Audio Tokens:

- **Universal:** Work well for audio, speech, and music without losing important information.
- **Compact:** Use low bit rates.
- **Efficient:** Fast encoding/decoding with minimal memory and compute overhead.
- **Streamable:** Support real-time, online processing.

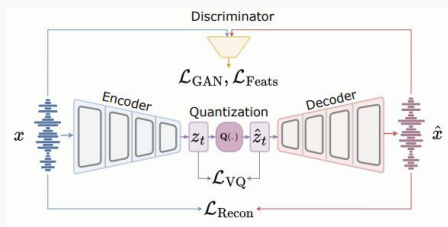


Audio Tokens

- Audio can be tokenized in different ways:

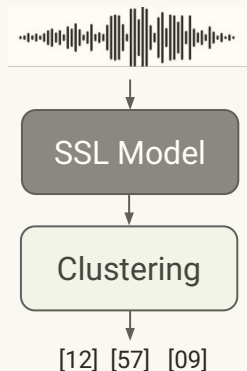
Acoustic Tokens (Audio Codecs)

Learned via an **autoencoder** with a **discrete bottleneck**.



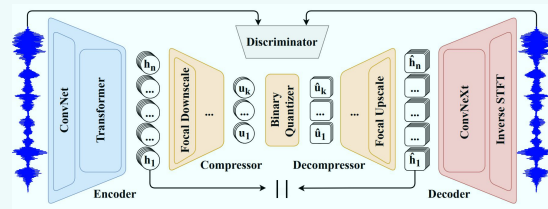
Semantic Tokens

Quantized representations from pretrained **self-supervised models**.



Hybrid Tokens

Integrate aspects of both **acoustic** and **semantic** tokens.



Acoustic Tokens

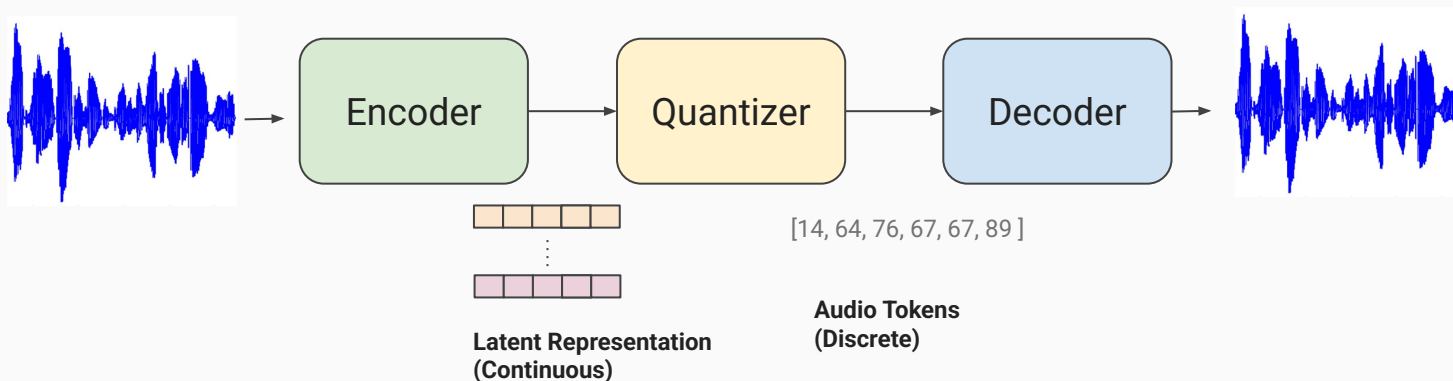
Acoustic Tokens

- A typical **acoustic tokenizer** consists of the following components:

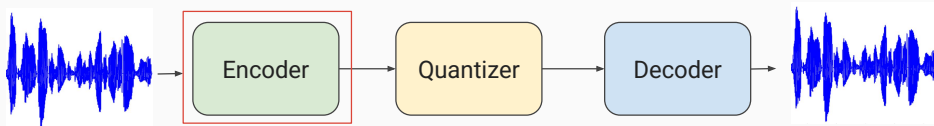
Encoder: Converts the input signal into a latent representation.

Quantizer: Transforms the continuous latent representation into discrete tokens.

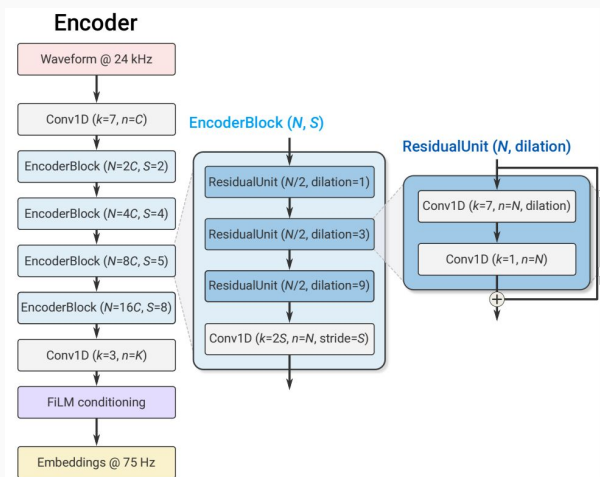
Decoder: Reconstructs the original signal from the tokens.



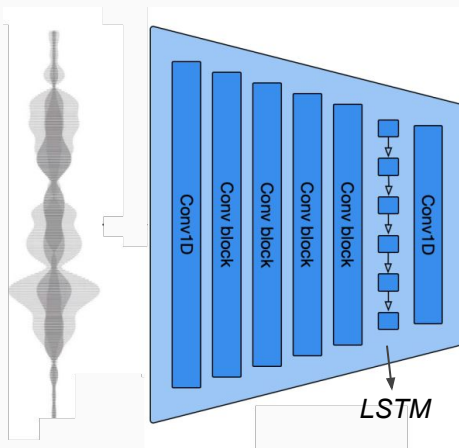
Encoder



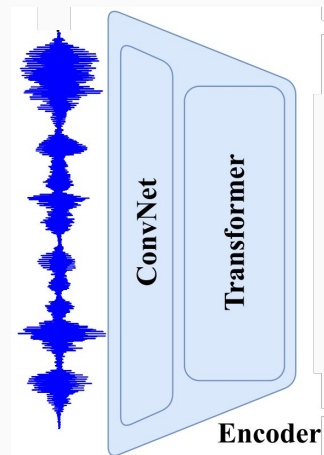
- The encoder is usually built on a **Convolutional Neural Networks (CNNs)**.
- **RNN** or **Transformer** layers are sometimes added to capture **long-term context**.



SoundStream (Zeghidour et al., 2021)

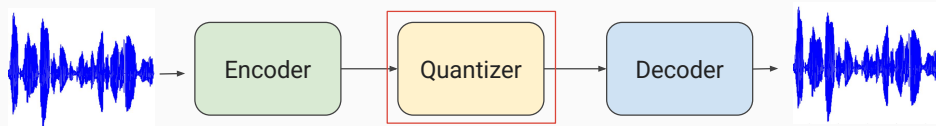


EncoDec (Défossez et al., 2022)

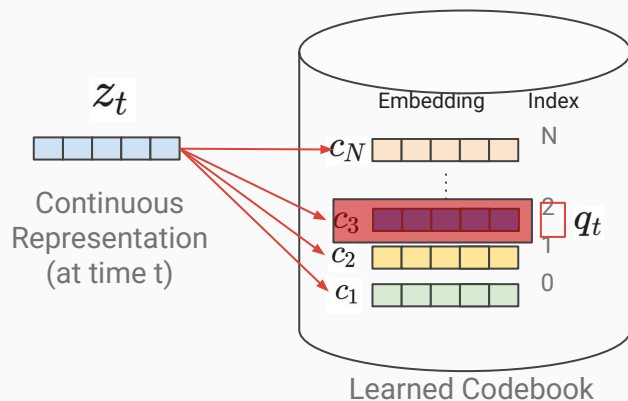


FocalCodec (DellaLibera et al., 2022) 28

Quantizer

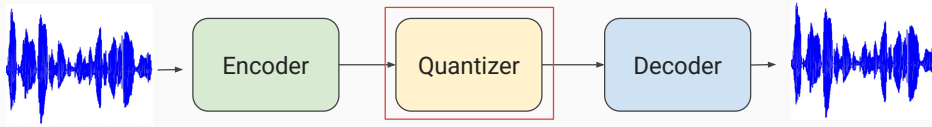


- The quantizer turns a **continuous** representation into a **discrete** one.
- **Basic Vector Quantization**: each input vector is assigned to the nearest codebook entry.



$$q_t = \arg \min_{i \in \{1, \dots, N\}} \|z_t - c_i\|^2$$

Quantizer

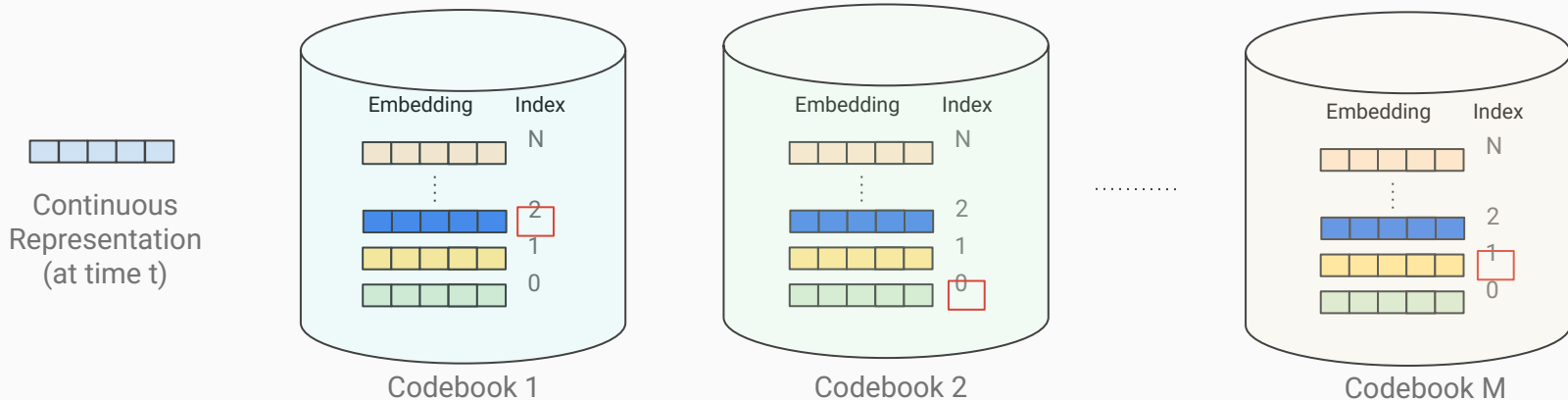


Problem

A **single codebook** (unless extremely large) cannot capture the full complexity of speech.

SOLUTION

We can use **multiple codebooks**!

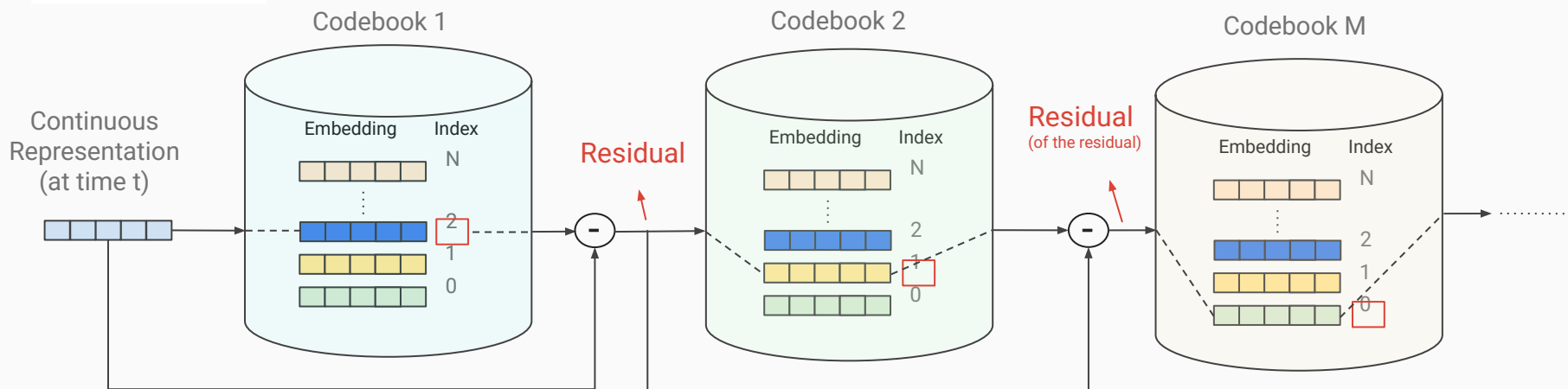


Residual Vector Quantization

- A convenient solution for combining multiple codebooks is **Residual Vector Quantization (RVQ)**.

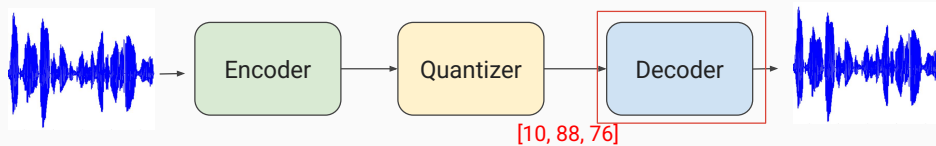


RVQ maps each feature to the nearest codebook entry, then quantizes the **residual** to refine the representation.

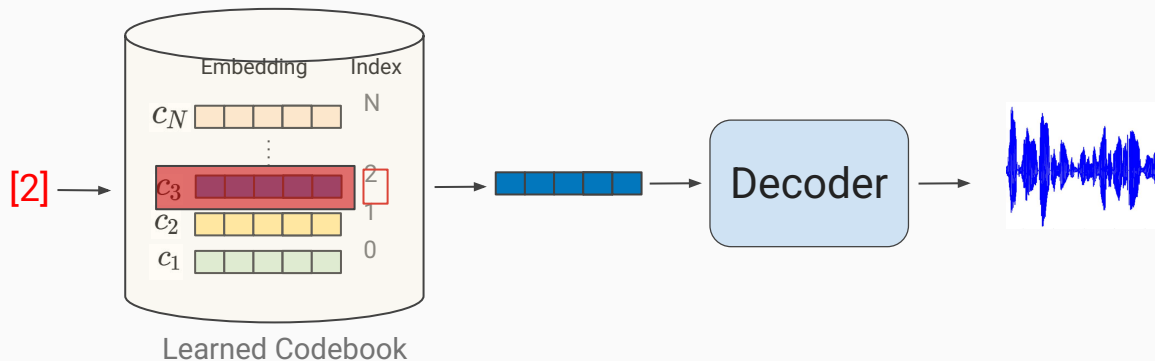


The first codebook encodes **coarse structure**, and subsequent codebooks **refine details**.

Decoder



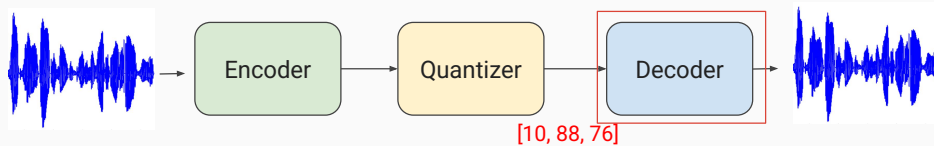
- The decoder **reconstructs** the **original waveform** from the discrete token indexes.



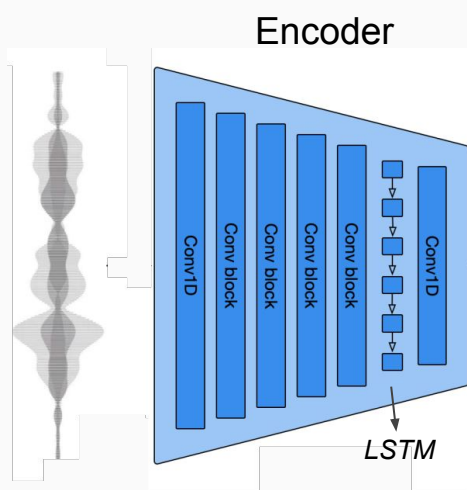
First, the codebook converts each token index into its continuous vector.

Then a neural network reconstructs the original signal.

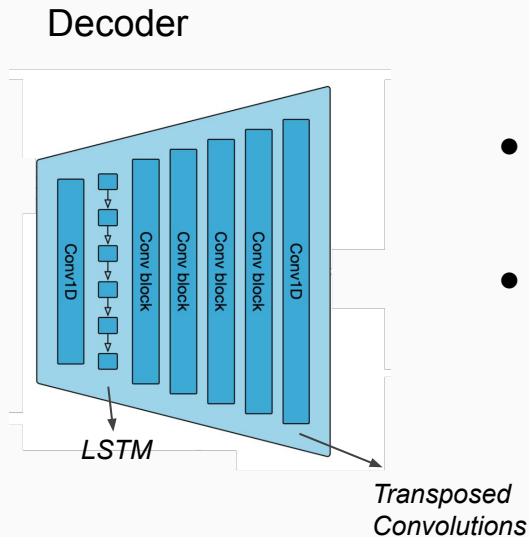
Decoder



- The decoder typically mirrors the encoder's architecture.

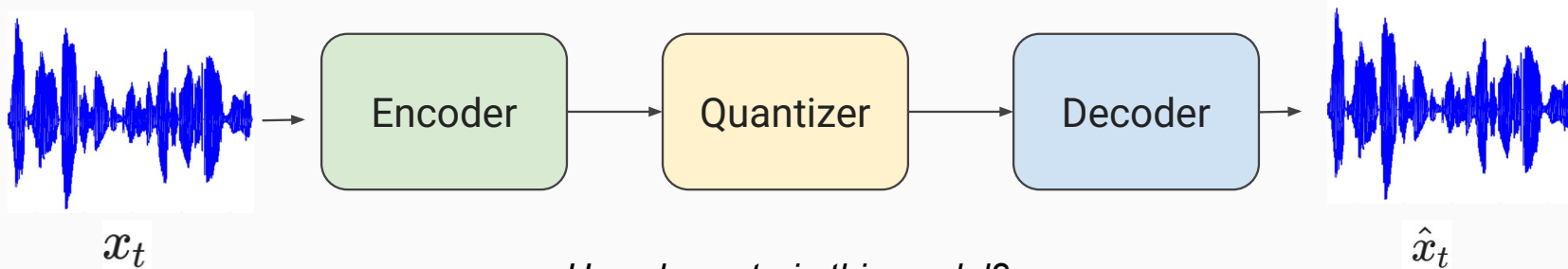


EncoDec (Défossez et al., 2022)



- The decoder has to **upsample** its input (to decompress).
- This is often done using **transposed convolutions**.

Training



How do we train this model?

- **Reconstruction loss:** train the model to reconstruct the original waveform from tokens:

$$L_{\text{Recon}} = \sum_{t=1}^T \|x_t - \hat{x}_t\|^2$$



Large-scale data are required (e.g, 60k hours of speech).

- **Standard training:** backpropagation + optimizers (e.g., Adam).



Unfortunately... it is not that simple....

Training

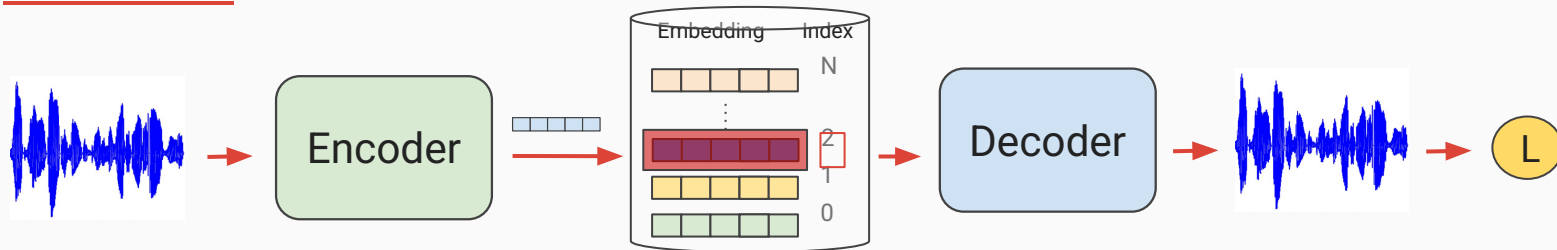
Problem

The model is **not end-to-end differentiable**.

SOLUTION

Use a **straight-through estimator**.

Forward Pass



- **Forward pass:** the input goes through the quantizer as usual.

Training

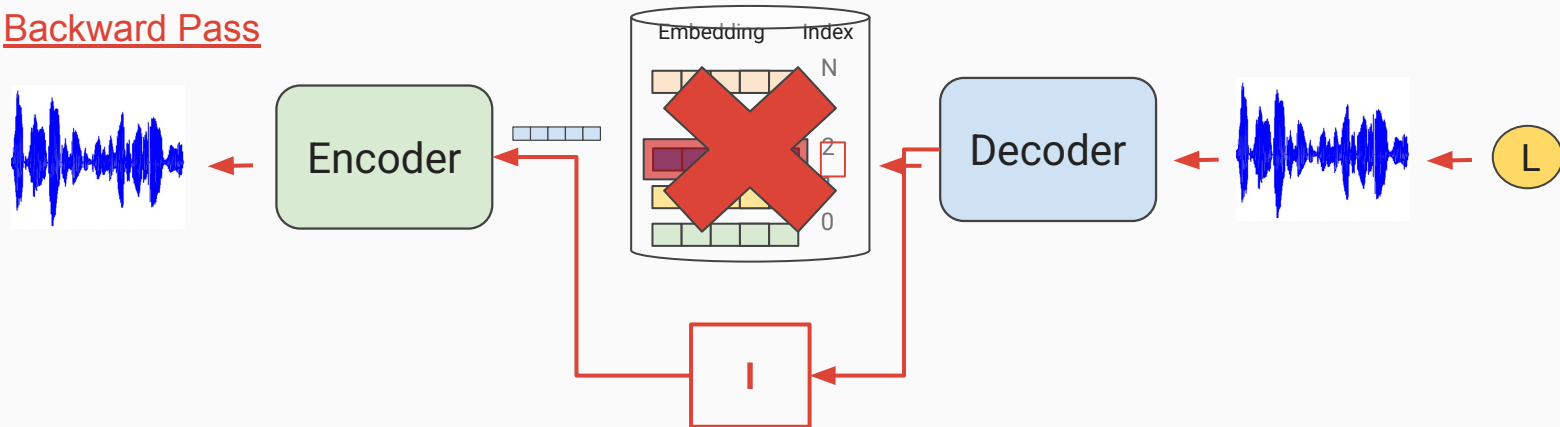
Problem

The model is **not end-to-end differentiable**.

SOLUTION

Use a **straight-through estimator**.

Backward Pass



- **Backward pass:** We **pretend** the quantizer is the **identity function** during backpropagation.
- During backprop, gradients flow from the decoder to the encoder, **bypassing** the **quantizer**.

Training

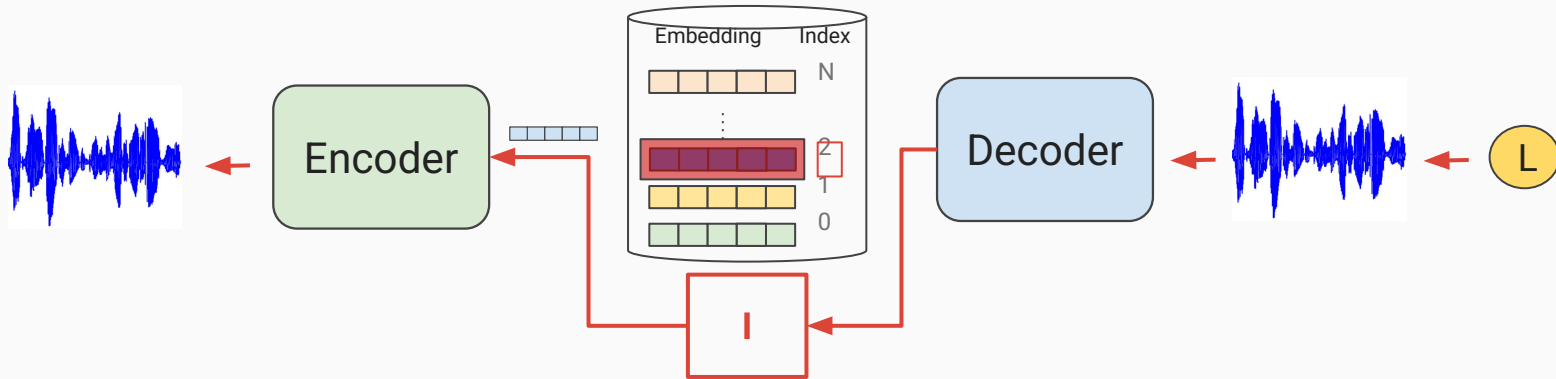
- The **straight-through** estimator is a rough approximation, but it **works well** in practice.

WHY?

- The encoder still receives **meaningful gradients**.
- Even if biased, the gradient still points to a **useful direction**.
- The encoder can thus learn representations that **minimize the loss**.

Problem

If gradients skip the quantizer, *how do we train the codebook?* **No gradients** → **no updates!**



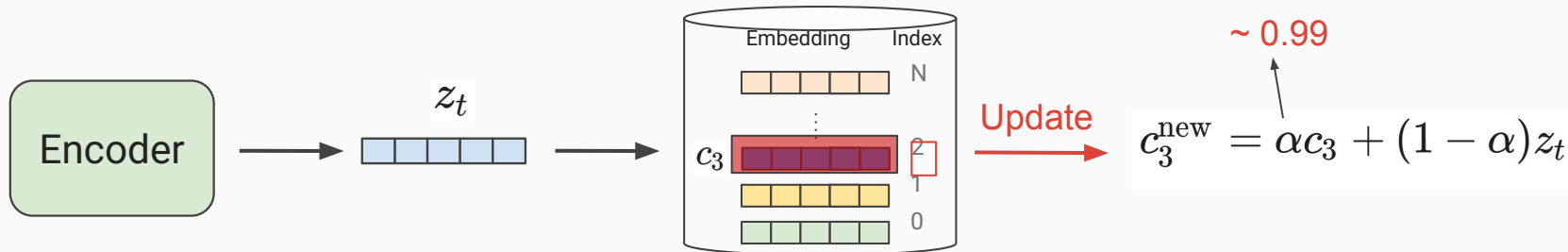
Training

Problem

How do we train the codebook?

SOLUTION

Each codebook vector is updated using an **exponential moving average (EMA)**.



First batch: Initialize all codebook entries from random batch vectors.

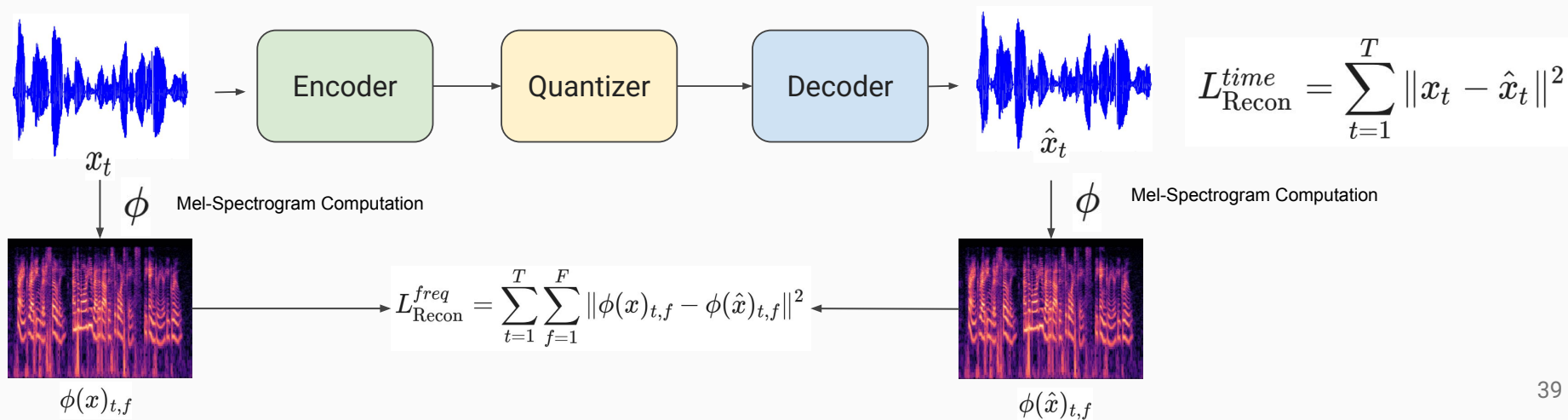
Later: Unused entries for many iterations are reinitialized from random encoded vectors in the current batch.

Training Tricks



- Additional **tricks** are often used to learn high-quality audio tokens.
- For instance, **multiple losses** are combined to improve performance.

Trick 1: Combine **time** and **frequency** reconstruction losses.

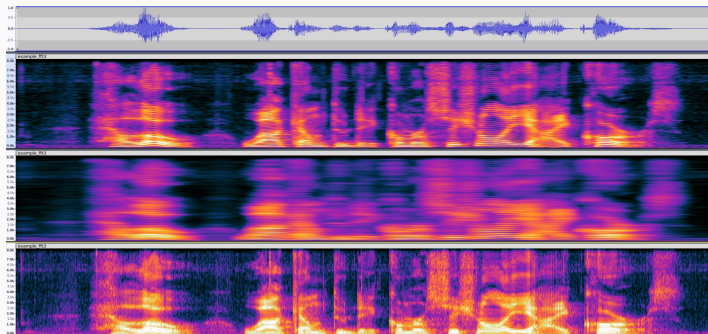


Training Tricks

Trick 1: Combine **time** and **frequency** reconstruction losses.

WHY?

Mel-spectrograms capture **perceptually relevant information** aligned with the human auditory system.



Waveform

Spectrogram
(wlen=1024)

Wideband Spectrogram
(wlen=4096)

Narrowband Spectrogram
(wlen=512)

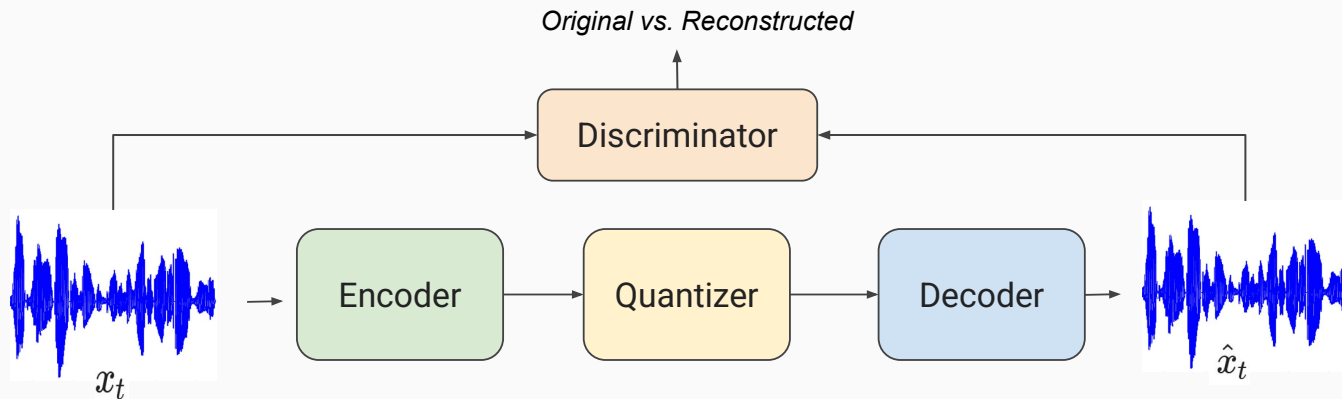
Additional Improvements:

- Use a weighted combination of **L1 and L2 losses** computed on mel-spectrograms.
 - Compute losses at **multiple resolutions** by varying the window length (and hop length).
- Multiple resolutions encourage the model to learn representations that capture both **fine time and frequency details**.

Training Tricks

Trick 2: Adversarial Loss

- Add a **discriminator** trained to distinguish **original** vs. **reconstructed audio** (and spectrograms).

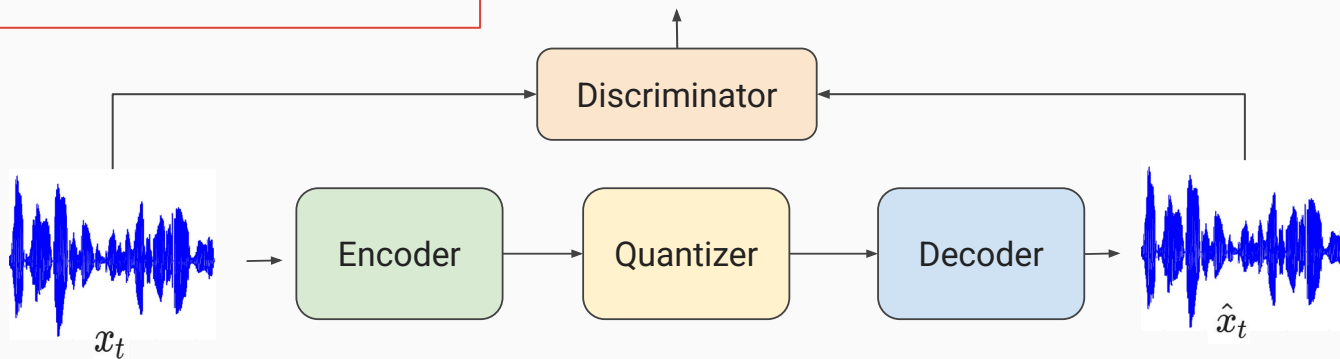


GAN-style training:

- The discriminator learns to detect artifacts in the reconstructed signals.
- The codec tries to fool the discriminator, producing more natural and realistic audio.

Training Tricks

Trick 2: Adversarial Loss



- Discriminator Loss:

$$L_D = \max(0, 1 - D(x_t)) + \max(0, 1 + D(\hat{x}_t))$$

- Generator Loss:

$$L_G = \max(0, 1 - D(\hat{x}_t))$$

Hinge Loss
GANs

Training Tricks

Trick 2: Adversarial Loss

WHY?

- Reconstruction losses (e.g., L1/L2) often produce **smooth outputs** and miss details.
- The adversarial loss helps capture **fine details** by pushing the decoder to generate more **realistic waveforms**.

Improvement

- **Multi-resolution discriminators:** operate on spectrograms at multiple time–frequency resolutions.

- **Feature Matching:** minimize the L1 distance between discriminator internal features for real and reconstructed audio.

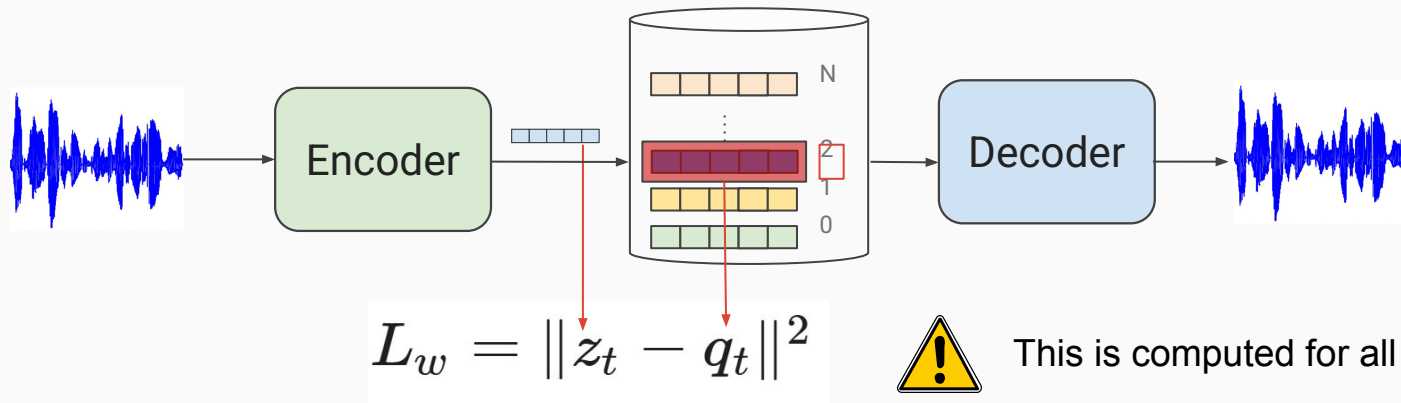
$$\ell_{feat}(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{KL} \sum_{k=1}^K \sum_{l=1}^L \frac{\|D_k^l(\mathbf{x}) - D_k^l(\hat{\mathbf{x}})\|_1}{\text{mean}(\|D_k^l(\mathbf{x})\|_1)}$$

Discriminators # Layers # Internal Representation

Training Tricks

Trick 3: VQ commitment loss

- Add a **VQ commitment loss** between the output of the encoder and its quantized value.



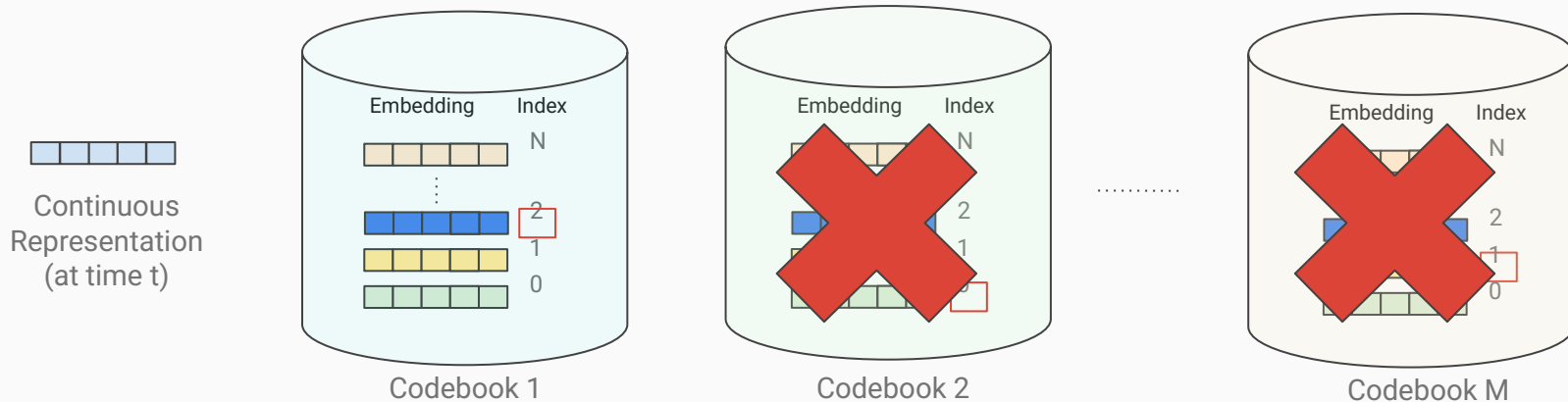
WHY?

The VQ commitment loss keeps the encoder outputs close to their assigned codebook vectors, preventing them from moving “too far” away.

Training Tricks

Trick 4: Multi-bandwidth training

- With Residual Vector Quantization, we **randomly select** how many residual **codebooks** to use during training.



WHY?

We can learn a single model that operates at **multiple compression levels**.

Performance



- With all these tricks, the model can be trained to achieve **competitive performance**!

Ground Truth

EncoDec (1.5 kbps)

EncoDec (3 kbps)

EncoDec (6 kbps)

EncoDec (12 kbps)



- There is a trade-off between **compression** and **reconstruction quality**.



We compress a lot!

- Raw Waveform: 256 kbps
- MP3: 32 kbps
- Opus: 6-24 kbps

[More Samples](#)

Other Acoustic Tokens

- In the previous slides, we mainly focussed on **SoundStream** and **EncoDec**.



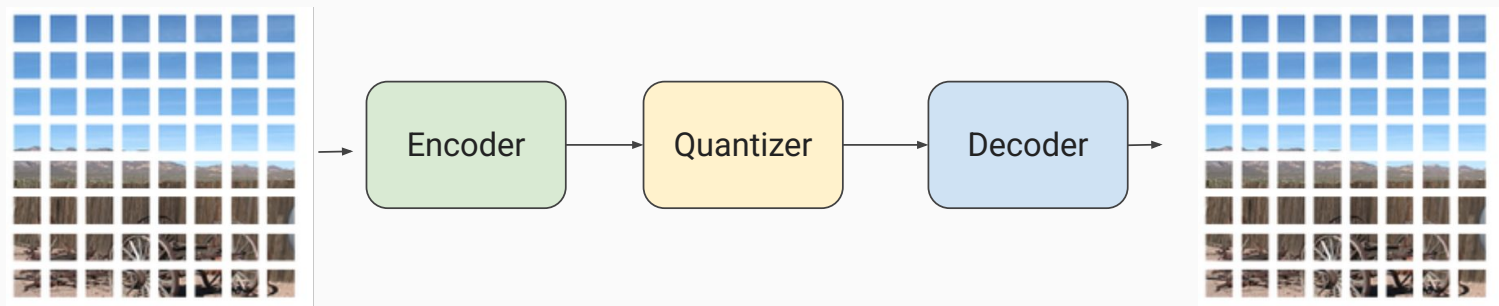
Many variants have been proposed!



APCodec (Ai et al., 2024)	HiFi-Codec (Yang et al., 2023a)	PAST (Har-Tuv et al., 2025)	SpeechTokenizer (Zhang et al., 2024a)
AudioDec (Wu et al., 2023b)	HILCodec (Ahn et al., 2024)	PQ-VAE (Guo et al., 2024b)	SQ-Codec (Yang et al., 2024d)
Best-RQ (Chiu et al., 2022)	LaDiffCodec (Yang et al., 2024e)	Prompt Codec (Pan et al., 2024)	TAAE (Parker et al., 2025)
BigCodec (Xin et al., 2024)	Language Codec (Ji et al., 2024b)	RepCodec (Huang et al., 2024)	TFNet (Jiang et al., 2022b)
DAC (Kumar et al., 2023)	LFSC (Casanova et al., 2025)	S-TFNet (Jiang et al., 2022a)	Ti-Codec (Ren et al., 2024b)
Discrete SSL (Mousavi et al., 2024b)	LLMCodec (Yang et al., 2024b)	S3 (Du et al., 2024a)	TS3-Codec (Wu et al., 2024d)
Disen-TF-Codec (Jiang et al., 2023)	LSCodec (Guo et al., 2025a)	SD-Codec (Bie et al., 2025)	USM (Zhang et al., 2023)
dMel (Bai et al., 2024)	MDCTCodec (Jiang et al., 2024)	SemantiCodec (Liu et al., 2024a)	Vocos (Siuzdak, 2024)
ESC (Gu & Diao, 2024)	Mimi (Défossez et al., 2024)	Single Codec (Li et al., 2024)	WavTokenizer (Ji et al., 2024c)
FACodec (Ju et al., 2024)	MMM (Shi et al., 2024b)	SingOMD (Tang et al., 2024b)	Wav2Vec-BERT (Chung et al., 2021)
FunCodec (Du et al., 2023)	NAST (Messica & Adi, 2024)	SNAC (Siuzdak et al., 2024)	WMCodec (Zhou et al., 2024)
HARP-Net (Petermann et al., 2021)	NDVQ (Niu et al., 2024)	SOCODEC (Guo et al., 2024a)	X-Codec (Ye et al., 2025)
		Spectral Codecs (Langman et al., 2024)	XEUS (Chen et al., 2024b)

Image Tokens

- Image tokens are computed similarly to audio tokens.



1. Split the image into **patches**.
2. **Encode** each patch using a model (e.g., ViT).
3. **Quantize** the encoded representations into discrete tokens.
4. **Decoder** reconstructs the original image from the tokens.

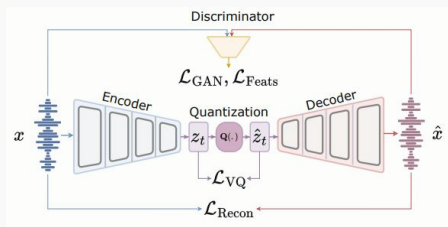
Semantic Tokens

Semantic Tokens

- Audio can be tokenized in different ways:

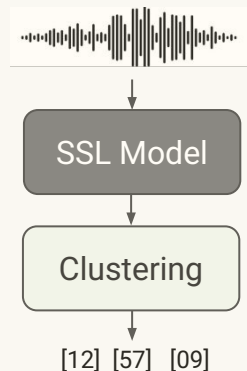
Acoustic Tokens (Audio Codecs)

Learned via an **autoencoder** with a **discrete bottleneck**.



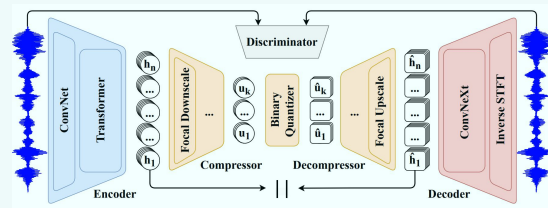
Semantic Tokens

Quantized representations from pretrained **self-supervised models**.



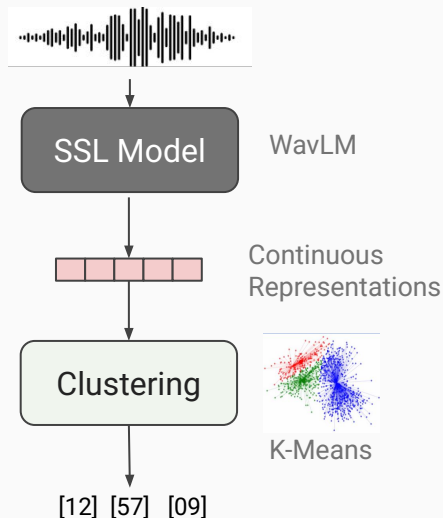
Hybrid Tokens

Integrate aspects of both **acoustic** and **semantic tokens**.



Semantic Tokens

- Unlike audio tokens, semantic tokens are not learned from scratch via an autoencoder.
- Instead, we quantize representations from **pretrained self-supervised models** like WavLM.



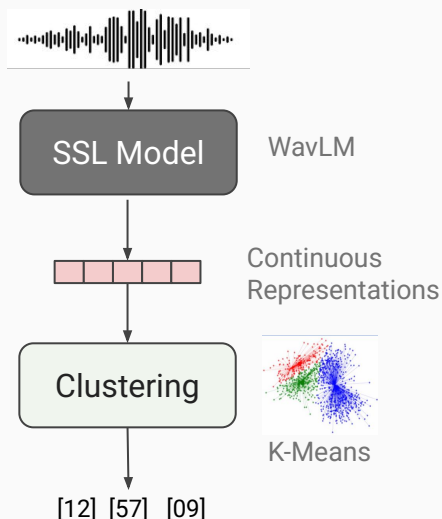
Why not use pretrained models that work great?

1. Extract **internal features** from a pretrained self-supervised model.
2. Cluster the features using an algorithm like **k-means**.
3. At inference, map each input feature to its **nearest centroid**.



Not Really....

Semantic Tokens



1. Which layers should we cluster?
2. What is the optimal number of clusters?
3. Which datasets are we using for clustering?
4. What is the best approach to train the decoder?
5. Can we extract universal tokens?



How Should We Extract Discrete Audio Tokens from Self-Supervised Models?

Pooneh Mousavi^{1,2}, Jarod Duret³, Salah Zaiem⁴, Luca Della Libera^{1,2}, Artem Ploujnikov^{5,2}, Cem Subakan^{6,2,1}, Mirco Ravanelli^{1,2,5}

With the right design choices,
semantic audio tokens are very
competitive!

Acoustic vs Semantic Tokens

Acoustic Tokens



End-to-end training.



High reconstruction quality.



Trained from scratch (costly and data-hungry).



Do not leverage large pretrained SSL models.

Semantic Tokens



Leverage strong pretrained SSL models.



Capture high-level semantic structure such as phonemes, words, and speaker info.



Poorer waveform reconstruction (not optimized for that).



Not end-to-end: requires a two-step pipeline (quantization → decoder training).



Can we have the best of both worlds?

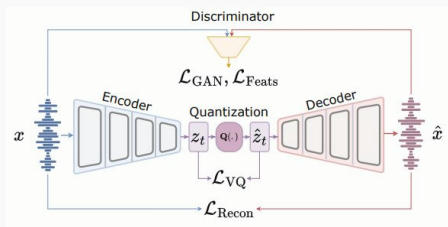
Hybrid Tokens

Hybrid Tokens

- Audio can be tokenized in different ways:

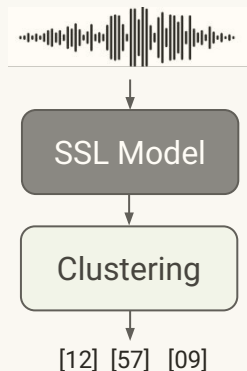
Acoustic Tokens (Audio Codecs)

Learned via an **autoencoder** with a **discrete bottleneck**.



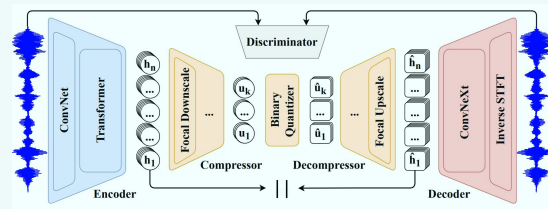
Semantic Tokens

Quantized representations from pretrained **self-supervised models**.

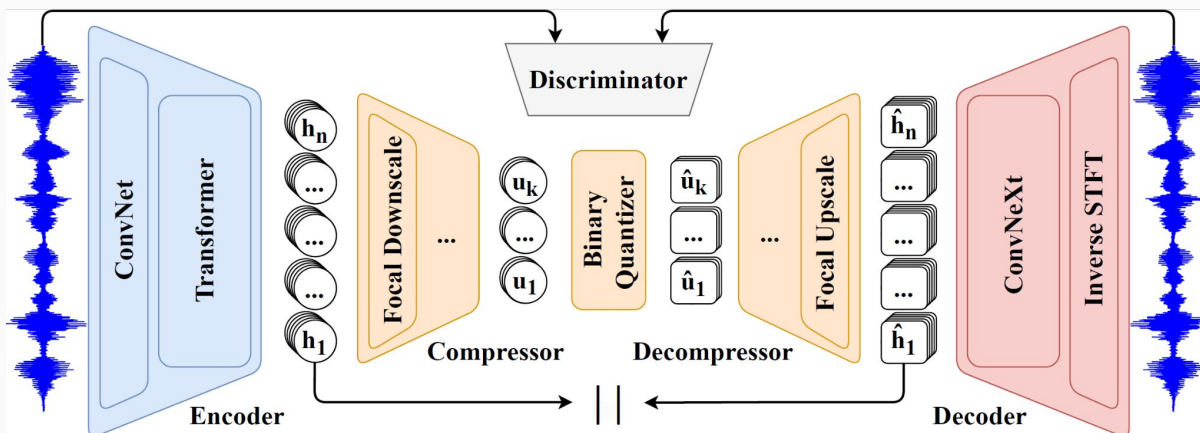


Hybrid Tokens

Integrate aspects of both **acoustic** and **semantic** tokens.



FocalCodec



- **Hybrid Representations:** Combines semantic features (WavLM) and acoustic features (via reconstruction).
- **Binary Spherical Quantization (BSQ):** Lookup-free, compact, and highly efficient (first time used in speech).
- **Ultra-Low Bitrates:** Compresses speech to 0.16–0.65 kbps with a single binary codebook.
- **Focal Modulation:** Replaces self-attention with focal modulation for fast, scalable encoding and decoding.

FocalCodec

Ground Truth



EncoDec
(1.5 kbps)



FocalCodec
(0.16 kbps)



FocalCodec
(0.33 kbps)



FocalCodec
(0.66 kbps)



[Training Code on SpeechBrain](#)

[Inference Code](#)

[Pretrained Model](#)

[Demo Samples](#)

FocalCodec: Low-Bitrate Speech Coding via Focal Modulation Networks

(Della Libera et al., NeurIPS 2025)

Luca Della Libera^{1,2} Francesco Paissan^{3,2} Cem Subakan^{4,1,2} Mirco Ravanelli^{1,2}

<https://arxiv.org/pdf/2502.04465>

Benchmarks

Benchmarks

- *How Do We Evaluate Audio Tokens?*

Ideally:

- Feed audio tokens into a multimodal LLM and evaluate end-task performance.
- **But:** training such models is *extremely expensive*.

What is done: Use Proxy Benchmarks

[Codec Superb](#)



Evaluates **reconstruction quality** (waveform fidelity, perceptual metrics).

[Discrete Audio and Speech Benchmark \(DASB\)](#)



Evaluates **downstream task performance** (speech recognition, speaker ID, Emotion ID, TTS, etc.)



Benchmarks

Published in Transactions on Machine Learning Research (09/2025)

Discrete Audio Tokens: More Than a Survey!

Pooneh Mousavi*^{†1,2}, Gallil Maimon*³, Adel Moumen*⁴, Darius Petermann*⁵, Jiatong Shi*⁶, Haibin Wu*⁷, Haici Yang*⁵, Anastasia Kuznetsova*⁵, Artem Ploujnikov^{8,2}, Ricard Marxer⁹, Bhuvana Ramabhadran¹⁰, Benjamin Elizalde¹¹, Loren Lugosch¹¹, Jinyu Li⁷, Cem Subakan^{12,2,1}, Phil Woodland⁴, Minje Kim¹⁴, Hung-yi Lee¹³, Shinji Watanabe⁶, Yossi Adi³, Mirco Ravanelli^{1,2,8}

¹Concordia University, ²Mila-Quebec AI Institute, ³The Hebrew University of Jerusalem, ⁴University of Cambridge, ⁵Indiana University, ⁶Carnegie Mellon University, ⁷Microsoft, ⁸Université de Montréal, ⁹Université de Toulon, ¹⁰Google, ¹¹Apple ¹²Laval University, ¹³National Taiwan University, ¹⁴University of Illinois at Urbana-Champaign



Large Gap vs. **Continuous Representations!**



Audio Tokenizers Are Not Universal (Yet!)

[Paper](#)

Open Challenges & Future Directions

Open Challenges

Ideal Audio Tokens:

- **Universal:** Work well for audio, speech, and music without losing important information.
- **Compact:** Use low bit rates.
- **Efficient:** Fast encoding/decoding with minimal memory and compute overhead.
- **Streamable:** Support real-time, online processing.



- We've made **significant progress**, but we're still **far** from the ideal requirements.
- Significant **research** is still needed.



Future Directions

- Universal Audio Tokens



Some Ideas

- **Massive Multitask** learning (similar to [PASE](#))
- **Hierarchical** codebooks with **Dynamic Allocation** (more details for Music)
- **Perceptual Loss Optimization** (Similar to [MetricGAN](#))
- Better **Multi-scale** processing.

Future Directions

- Can we learn “**interpretable**” audio tokens?

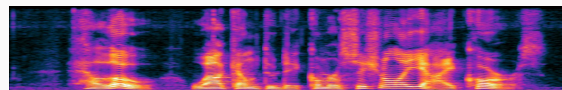
Text: “The City of Montréal”

Tokenized: “[The] [City] [of] [Mont] [ré] [al]”



Each token is **easily interpretable**, as it maps to a specific part of the text.

Audio



Tokenized Audio

[56]	[12]	[26]	[18]
[73]	[57]	[23]	[32]
[38]	[09]	[78]	[61]



No clear mapping to the original signal

Future Directions

Do we really need tokenization?

- Tokenization always loses some information.
- Future multimodal LLMs might work directly with **continuous representations**.
- We may need architectures **beyond pure autoregression**, e.g., combining **diffusion** and **autoregressive models**.



Thank you!

